Dynamic Queuing Model for Efficient Instruction Scheduling and Resource Allocation in Tomasulo's Algorithm

Hema Kanoujia
Harper Archambault
Lauren Caccamise
Lakshmi Katravulapalli
Tien-Thanh Bui

April 14, 2025

Abstract

Processors feature a variety of functional units (FUs), each specialized for different types of operations, such as integer arithmetic, floating-point computation, or memory access, with varying execution speeds and resource costs. Therefore, adjusting these functional units' operating rates or resource allocation is essential to achieving optimal performance. However, efficient scheduling of instructions across these diverse FUs is challenging due to data hazards, instruction dependencies, and resource conflicts. To address these challenges, the Tomasulo algorithm was introduced as an out-of-order execution mechanism that enhances instruction-level parallelism by dynamically scheduling instructions based on data availability rather than static program order. While previous research has focused on the Tomasulo algorithm's ability to improve instruction throughput through dynamic scheduling and out-of-order execution, as well as the application of queuing theory to optimize performance, most approaches were based on static resource allocation. This work proposes a cost allocation model for the functional units of the Tomasulo algorithm, in which the functional units are modeled as queuing systems that dynamically adjust service rates or the reallocation of resources based on real-time factors. By combining dynamic scheduling with adaptive resource management, this research is expected to reduce the average instruction wait time compared to static resource allocation. The findings will contribute to a more adaptive and higher-performing approach to resource management in Tomasulo-like architectures, influencing future studies in computer architecture and dynamic scheduling.

1 Background

Modern processors are designed to support high levels of instruction-level parallelism (ILP), allowing multiple instructions to execute simultaneously by leveraging a wide range of spe-

cialized functional units (FUs). These FUs are optimized for different classes of operations, such as integer arithmetic, floating-point computation, and memory access, each with distinct execution latencies and hardware resource requirements. To maximize performance, effective utilization and allocation of these FUs is essential.

However, efficient scheduling of instructions across diverse FUs presents significant challenges due to the presence of data hazards, instruction dependencies, and resource conflicts. Static scheduling, in which instructions execute in the order specified by the program, often leads to pipeline stalls and underutilization of available resources. To address these limitations, out-of-order execution mechanisms have been introduced to dynamically reorder instructions at runtime based on data availability and resource readiness, rather than program order.

A landmark development in this space is the Tomasulo algorithm[1], first introduced in IBM's 360/91 architecture. The algorithm employs dynamic scheduling, register renaming, and reservation stations to resolve hazards and allow for out-of-order execution, thus significantly improving ILP. Tomasulo's approach enables instructions to proceed as soon as their operands are available, decoupling execution from program order and reducing idle cycles in functional units.

Despite these advances, much of the existing research has focused on the static allocation of FUs, where service rates and resource availability remain fixed during execution. While some studies have modeled Tomasulo-like architectures using queuing theory to evaluate and improve performance, the potential of dynamic resource allocation in this context remains underexplored.

In this work, we build upon these foundations by introducing a cost-aware, adaptive resource management model for Tomasulo-style architectures. In our approach, each FU is modeled as a queuing system with dynamically adjustable service rates or reconfigurable resources, responsive to real-time execution characteristics. By integrating dynamic FU management with the Tomasulo algorithm's inherent scheduling flexibility, the proposed model aims to reduce instruction wait times and enhance overall processor efficiency.

The remainder of this work will explore the implications of adaptive FU allocation on ILP and performance metrics, offering a novel perspective on scheduling and resource optimization in modern microarchitectures.

2 Method

The motivation for our new project idea began with the problem posed by Tomasulo's algorithm, which is that it fails to efficiently handle changing workloads. Fixed functional unit assignments frequently cause issues such as underutilization, resource bottlenecks, and resource overloading. This can lead to an increase in execution time and a decrease in the effectiveness of out-of-order instruction handling. The proposed solution is to implement a dynamic queuing model that can provide flexible service rates and resource allocation in real-time. We can introduce a feedback mechanism that allows the system to track workload fluctuations and efficiently reallocate system resources based on real-time demands. This method can optimize instruction scheduling, lower execution stalls, and improve resource allocation. In addition, it can ensure that system resources are distributed accordingly,

leading to improved throughput and performance for out-of-order execution situations. By utilizing the dynamic feedback method, it will contribute to a more scalable and effective approach to Tomasulo's algorithm.

2.1 Mathematical Model

Our work models each FU as an independent M/D/1 queue [2], where:

- (M) Instruction arrivals follow a Poisson process with rate λ_i for FU i
- (D) Service times are deterministic with rate μ_i (instructions/ns)
- (1) Each FU processes one instruction at a time in a FIFO order

The average waiting time W_i for instructions in FU i is given by:

$$W_i = \frac{1}{\mu_i} + \frac{\rho_i}{2\mu_i(1-\rho_i)}, \quad \rho_i = \frac{\lambda_i}{\mu_i} \tag{1}$$

where ρ_i is the utilization rate (stability requires $\rho_i < 1$). The total waiting time is weighted by arrival rates and scaled by k (instructions/cycle). The optimization objective minimizes the total weighted waiting time:

$$\sum_{i=1}^{n} \lambda_i W_i \tag{2}$$

subject to:

- Budget constraint: $\sum_{i=1}^{n} a_i \mu_i = C$, where a_i = cost coefficient for FU i, C = total budget
- Stability constraint: $\mu_i > \lambda_i \ \forall i$

For dynamic adaptation, arrival rates λ_i are continuously estimated using exponential moving averages (EMA) of observed instruction counts over optimization windows. EMA is represented as follows:

$$\lambda_i^{\text{new}} = \alpha \cdot (\frac{\text{Count}_i}{\text{Window Size}}) + (1-\alpha) \cdot \lambda_i^{\text{old}} \tag{3}$$

The model also uses sequential least squares programming (SLSQP) for performing optimization for both static and dynamic. The dynamic version will reoptimize every 1000 cycles using warm starting.

2.2 Optimizing the model

We expand upon Yang's static method as follows. Their approach solves the optimization problem once (one-time optimization), uses the expected arrival rates λ_i, and sets the initial guess as $\mu_i = \alpha_i \times \frac{\lambda_i}{a_i}$ with α_i as proportional coefficients. It assumes a stable instruction mix, resulting in lower overhead but less responsiveness to dynamic changes.

Our dynamic method, in contrast, re-optimizes every 1000 cycles and updates λ_i in real-time using an Exponential Moving Average. It warm-starts the optimization from the previous μ_i solution, allowing it to better handle fluctuating workloads. Although this introduces slight overhead, it achieves higher accuracy over time.

3 Experiments and implementation details

Our Python implementation models processor performance by optimizing Tomasulo's algorithm based on queuing theory. The code provides two approaches regarding resource allocation across functional units within the processor, static and dynamic. The static approach computes fixed optimal resource allocations while the dynamic approach continuously adapts to the changing workloads. Both approaches follow the M/D/1 queuing system and constrained optimization (SLSQP) that was originally used in Yang et al.'s 2024 study. The code determines the cost allocations, service rates, average waiting time, and utilization of both approaches, then compares their abilities to minimize instruction waiting times under budget constraints.

Input

In order to run the simulator, four items were necessary that would be applied to both the static and dynamic approaches. The input consists of the number of instructions sent in each cycle (k), the total cost of FU (C), the instruction arrival rate (instruction/ns) ($\bar{\lambda}$), and the cost coefficient of FU ($\bar{a}$). The following table displays the input for each example:

Table 1: Simulator Input

Examples	**Example 1** (Low intensity)	**Example 2** (Medium intensity)	**Example 3** (High intensity)
Number of instructions sent in each cycle k	1	3	5
Total cost of functional units C	200	500	700
Instruction arrival rate (instructions/ns) $\bar{\lambda} = [\lambda_1, \lambda_2, \lambda_3, \lambda_4]$	[0.1,0.3,0.4,0.2]	[0.2,1.3,1.2,0.3]	[0.5,1.7,2.0,0.8]
Cost coefficient of functional units $\bar{a} = [a_1, a_2, a_3, a_4]$	[5,30,200,10]	[5,30,200,10]	[5,30,100,10]

StaticTomasulo Class

The first class in the program applies the static approach that replicates the results featured in the original work. It computes the optimal fixed allocation based on the expected work-

load. This class is composed of three methods including `__init__`, `objective_function`, and `solve_optimal_allocation`.

- `__init__(self, cost_coefficients, arrival_rates, total_cost, k)`: Initializes cost coefficients (a_i), arrival rates (λ_i), total budget (C), and instructions per cycle (k).
- `objective_function(self, mus)`: Computes the total weighted waiting time for the given service rates (μ_i).
- `solve_optimal_allocation(self)`: Minimizes the total waiting time subject to budget constraint $\sum a_i \mu_i = C$ and stability constraints.
  - Initial guess: Allocates μ_i proportional to λ_i / a_i
  - Constraints: Enforces budget and stability through SciPy's SLSQP
  - Objective: Minimizes $\sum \lambda_i W_i$ (weighted waiting time)

DynamicTomasulo Class

The next class applies the dynamic approach by continuously adjusting the functional unit allocations based on real-time instruction arrivals. Some key features included in this class include Exponential Moving Average (EMA) for smoothing arrival rates, periodic re-optimization, and warm-starting using the previous μ_i as the initial guess. The methods that are utilized in this class include `__init__`, `optimize`, `_objective_function`, `log_instruction`, `get_current_allocation`, and `plot_optimization_history`.

- `__init__(self, cost_coefficients, initial_arrival_rates, total_cost, k)`: Initializes cost coefficients (a_i), arrival rates (λ_i), total budget (C), and instructions per cycle (k).
- `optimize(self)`: Recomputes the optimal μ_i using the latest workload statistics.
  - Updates arrival rates using EMA: $\lambda_i^{\text{new}} = \alpha \cdot (\frac{\text{Count}_i}{\text{Window Size}}) + (1 - \alpha) \cdot \lambda_i^{\text{old}}$
  - Recompute μ_i using warm-starting optimization
- `_objective_function(self, mus)`: Computes the total weighted waiting time for the given service rates (μ_i).
- `log_instruction(self, inst_type)`: Processes a single instruction and triggers optimization at window boundaries.
  - EMA smoothing balances responsiveness and stability
  - Warm-starting uses previous μ_i to accelerate convergence
- `get_current_allocation(self)`: Returns the latest optimized functional unit allocation and performance metrics after optimization.
- `plot_optimization_history(self)`: Plots how μ_i and the waiting times evolve using a line graph.

4 Results

The results of the experiment demonstrate significant improvements regarding performance when using dynamic optimization compared to static allocation for the three workload scenarios. For the low-intensity workload (Figure 1), dynamic optimization reduced waiting times from 1.4323 ns to 1.2799 ns by reallocating resources from the floating-point multiplication unit to the integer and memory operations, resulting in a 10.6% reduction.

The medium-intensity workload (Figure 2) resulted in more dramatic improvements, with the waiting times being reduced from 2.0265 ns to 1.306 ns, allowing for a 35.55% latency reduction. This output resulted from a significant redistribution of resources from the floating-point multiplication units to the floating-point addition units.

Finally, the high-intensity workload (Figure 3) had a 20.6% latency reduction with waiting times decreasing from 1.2361 ns to 0.9808 ns by preventing any single FU from becoming a bottleneck through keeping all utilization rates below 11%.

The results from each test case validate the M/D/1 queuing model from Yang and Wu's original paper, specifically how dynamically adjusting service rates control queue utilization to minimize the waiting times. The consistent reduction in waiting times ranging from 10.6% to 35.5% through the application of dynamic optimization supports the benefit of real-time resource adaptation in processor scheduling, while still maintaining a fixed budget constraint.

It is important to note that these results may vary depending on the underlying hardware or across different runs, particularly due to the variability in workload characteristics and runtime conditions. Therefore, the work verifies that optimizing runtime based on real-time workload characteristics, instead of static assumptions, can enhance the performance of Tomasulo-based architectures.

Table 2: Example 1 (Low intensity) Results

	Static	**Dynamic**
Cost allocation of functional units	[6.9, 33.68, 145.04, 14.38]	[7.77, 36.97, 139.5, 15.75]
Execution rate of functional units (instructions/ns)	[1.38, 1.12, 0.73, 1.44]	[1.55, 1.23, 0.7, 1.57]
Average waiting time of instructions (ns)	1.4323 ns	1.2799 ns

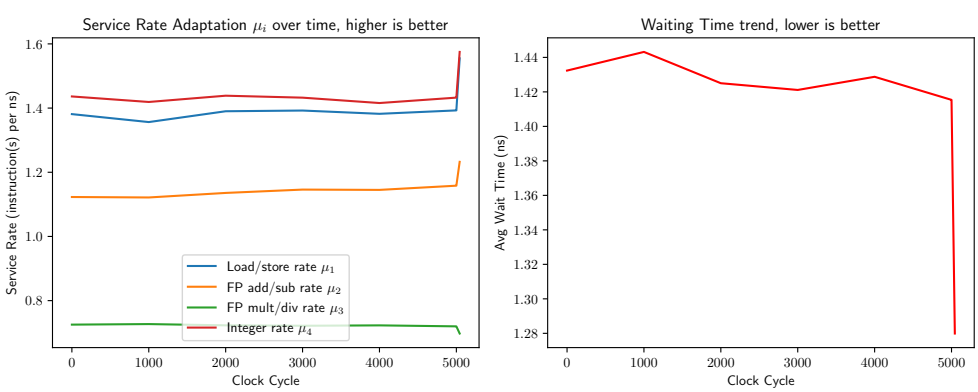

Figure 1: Example 1 - Low intensity load service rate adaptation and waiting time trend

Table 3: Example 2 (Medium intensity) Results

	Static	**Dynamic**
Cost allocation of functional units	[11.76, 96.74, 370.39, 21.12]	[18.98, 122.54, 325.49, 32.99]
Execution rate of functional units (instructions/ns)	[2.35, 3.22, 1.85, 2.11]	[3.8, 4.08, 1.63, 3.3]
Average waiting time of instructions (ns)	2.0265 ns	1.306 ns

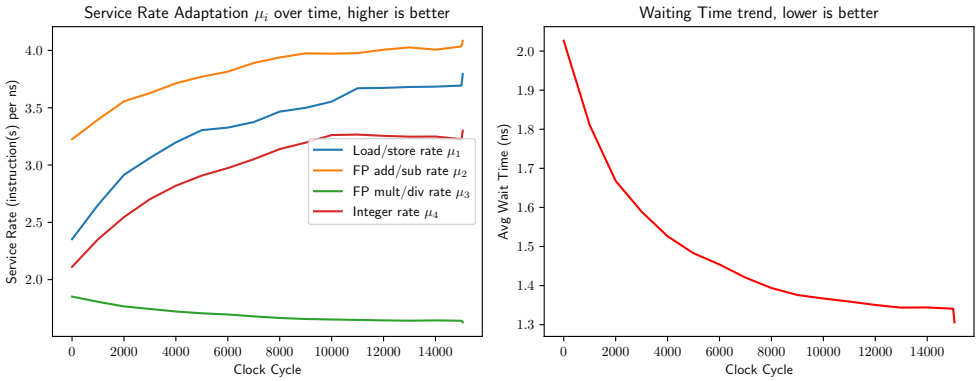

Figure 2: Example 2 - medium intensity load service rate adaptation and waiting time trend

Table 4: Example 3 (High intensity) Results

	Static	**Dynamic**
Cost allocation of functional units	[33.27, 176.15, 429.02, 61.56]	[41.42, 191.54, 392.32, 74.72]
Execution rate of functional units (instructions/ns)	[6.65, 5.87, 4.29, 6.16]	[8.28, 6.38, 3.92, 7.47]
Average waiting time of instructions (ns)	1.2361 ns	0.9808 ns

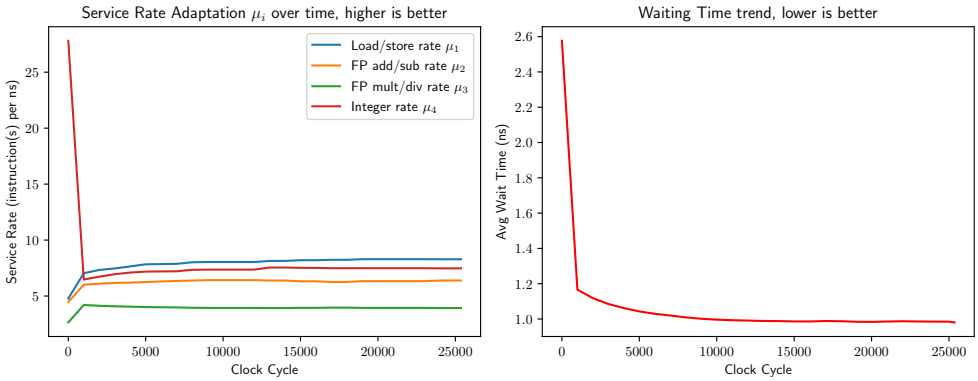

Figure 3: Example 3 - high intensity load service rate adaptation and waiting time trend

5 Comparison & Conclusion

The experimental results that were discussed in Chapter 4 highlight the effectiveness of dynamic resource allocation in improving the system performance of Tomasulo's algorithm.

When analyzing the different workload scenarios, we see that the low-intensity workload shows a latency reduction of 10.6%, while the medium-intensity workload showed the most significant reduction of 35.5%, along with the high-intensity workload, which had a balanced latency reduction of 20.6% overall. After comparing the static and dynamic models, we can see consistent reductions in the average waiting time of instructions across varying workloads. These results highlight how the dynamic model assigns functional units in a more efficient manner, along with reducing bottlenecks and underutilization. Our findings confirm our new approach: integrating dynamic queueing as well as feedback-based reallocation mechanisms with Tomasulo's algorithm can robustly reduce instruction waiting time and enhance functional unit utilization, contributing to higher overall performance and adaptivity with changing workload environments. In this paper, we have discussed and proposed our research idea, our code implementation, and experimental results. This presents potential for future research in dynamic resource allocation and out-of-order execution frameworks.

References

[1] R. M. Tomasulo. "An Efficient Algorithm for Exploiting Multiple Arithmetic Units". In: *IBM Journal of Research and Development* 11.1 (1967), pp. 25–33. DOI: 10.1147/rd.111.0025.

[2] T. Yang, Z. Wu, W. Chen, and H. Long. "Performance Optimization Model of Tomasulo Algorithm Based on Queuing Theory". In: *2024 7th International Conference on Advanced Algorithms and Control Engineering (ICAACE)*. 2024, pp. 101–106. DOI: 10.1109/ICAACE61206.2024.10548140.